

Pipeline di estrazione strutturata da scansioni — specificativa operativa

Target hardware: Ryzen 9 7950X, RTX 4090 (24GB), 32GB RAM. Output: JSON nidificato conforme a schema, con citazione verificabile per ogni valore.

Fase 0 — Preparazione

0.1 Schema come fonte unica di verità

Definisci lo schema in Pydantic v2. Da lì genera **due** varianti:

SchemaLoose → passata al decoder come `guided_json`. Solo tipi e struttura. Niente `pattern`, niente `ge`/`le`, tutto `Optional`. Grammatica semplice = compilazione veloce e nessun caso patologico in xgrammar.

SchemaStrict → usata per la validazione a valle. Tutti i vincoli, i validator custom, le regole cross-field.

Ogni campo foglia non è uno scalare, ma un oggetto con provenienza:

```
python
class Extracted[T](BaseModel):
    value: T | None
    quote: str | None # verbatim dal testo OCR
    page: int | None
    bbox: tuple[float, float, float, float] | None
    confidence: Literal["high", "low"]

    @model_validator(mode="after")
    def quote_required_with_value(self):
        if (self.value is None) != (self.quote is None):
            raise ValueError("value e quote devono essere entrambi nulli o entrambi presenti")
        return self
```

Questa è la struttura che rende possibile tutto il resto. Senza `quote` non hai verifica; senza `bbox` non hai revisione umana efficiente.

0.2 Gold set

30–50 documenti, scelti per essere **rappresentativi dei casi brutti**: pagine storte, timbri sovrapposti al testo, tabelle spezzate tra due pagine, scansioni sottoesposte, fotocopie di fotocopie.

Annotali a mano **prima** di scrivere i prompt. Se li annoti dopo aver visto l'output del modello, li annoterai inconsciamente per compiacerlo.

Split: ~15 documenti di sviluppo (li guardi liberamente), il resto **sigillato** fino alla valutazione finale.

0.3 Storage

SQLite in modalità WAL. Schema minimo:

```
sql
documents(doc_id PK, path, sha256, n_pages, status, created_at)
pages(doc_id, page_no, image_path, dpi, deskew_angle, full_text)
regions(region_id PK, doc_id, page_no, bbox, region_type, text, engine, order_idx)
region_conflicts(region_id, text_a, text_b, resolved_text, resolver)
extractions(doc_id, field_path, value_json, quote, page_no, bbox, attempt, status)
runs(run_id PK, git_sha, model, prompt_version, started_at)
```

`documents.status` è una macchina a stati:

```
ingested → rasterized → ocr_a → ocr_b → reconciled
          → enumerated → extracted → validated → done
                        ↘ needs_review
                        ↘ failed
```

Regola non negoziabile: ogni fase legge lo stato e salta ciò che è già fatto. Devi poter uccidere il processo in qualsiasi momento e ripartire senza perdere lavoro.

Fase 1 — Ingestione e rasterizzazione

sha256 del file sorgente → `doc_id`. Deduplica: se lo hai già visto, salta.

PyMuPDF: `page.get_pixmap(dpi=300)`. Per scansioni molto degradate prova 400 DPI, ma **misuralo**: oltre una certa soglia il rumore cresce più del segnale e l'OCR peggiora.

Deskew: stima l'angolo con trasformata di Hough sulle linee di testo, oppure `cv2.minAreaRect` sul contorno del testo binarizzato. Ruota solo se `|angolo| > 0.3°` — sotto quella soglia l'interpolazione fa più danni della correzione.

Normalizzazione: grayscale + `cv2.createCLAHE` per l'illuminazione irregolare. **Non binarizzare in modo**

aggressivo: i VLM moderni leggono meglio il grayscale che il bitonale, al contrario dei vecchi OCR.

Salva PNG. **Registra l'angolo di deskew applicato** — ti servirà per riproiettare le bbox sull'immagine originale.

Punto di controllo umano. Apri 20 pagine a campione e guardale davvero. È l'unico modo per accorgersi che il preprocessing sta cancellando timbri chiari o saturando le tabelle.

Fase 2 — OCR primario (PaddleOCR-VL-1.6)

```
bash
```

```
vllm serve PaddlePaddle/PaddleOCR-VL-1.6 \
--trust-remote-code \
--max-num-batched-tokens 16384 \
--no-enable-prefix-caching \
--mm-processor-cache-gb 0
```

```
python
from paddleocr import PaddleOCRVL
pipeline = PaddleOCRVL(
    pipeline_version="v1.6",
    vl_rec_backend="vllm-server",
    vl_rec_server_url="http://127.0.0.1:8080/v1",
)
```

Una `predict()` per pagina → regioni con tipo (testo / tabella / formula / timbro), bbox, testo, ordine di lettura.

Riproietta le bbox sulle coordinate dell'immagine originale invertendo la rotazione di skew.

Le tabelle vanno conservate come struttura, non appiattite in testo. PaddleOCR-VL le restituisce già in forma strutturata: in fase di estrazione una tabella ben serializzata vale dieci righe di prosa.

Scrivi in `regions`. Concatena il testo della pagina in ordine di lettura in `pages.full_text` — è il riferimento per il fuzzy match della Fase 6.

Fase 3 — Secondo OCR e riconciliazione

Spegni il server, avvia **DeepSeek-OCR 2** (o dots.ocr). `engine='b'`.

Allineamento delle due segmentazioni, che non coincideranno:

primo passo per IoU delle bbox, soglia 0.5;

le regioni rimaste non appaiate si allineano per contenuto con `rapidfuzz`.

Confronto: similarità normalizzata (lowercase, spazi collassati, punteggiatura unificata).

`≥ 0.95` → accetta il testo del motore A.

`< 0.95` → conflitto.

Risoluzione del conflitto: ritaglia la bbox unione con ~10px di margine, upscale 2×, rileggi con un terzo passaggio mirato a piena risoluzione. Maggioranza 2-su-3. Se tutti e tre divergono → `confidence='low'` e coda umana.

Attese realistiche: i conflitti si concentrano su cifre, codici alfanumerici e nomi propri. Se superano il ~5% del testo, il problema è a monte nella Fase 1, non nell'OCR.

Fase 4 — Enumerazione delle liste

Questa fase esiste perché lo schema è nidificato. **Saltarla è la causa numero uno di elementi mancanti o duplicati.**

Per ogni campo di tipo lista, una chiamata dedicata che restituisce **solo** l'inventario:

```
json
{"items": [{"anchor": "Contratto n. 44/B", "page": 3, "region_ids": [12, 13]}]}
```

`anchor` è una stringa verbatim che identifica univocamente l'inizio dell'elemento.

Verifica ogni anchor contro il testo OCR. Anchor non trovata → elemento scartato (era inventato).

Controllo di copertura indipendente: conta le righe della tabella, le occorrenze di un pattern di numerazione, o le intestazioni di sezione. Se il conteggio non torna con la lista enumerata, il documento va in revisione. **Non procedere in silenzio.**

Deduplica per anchor normalizzata.

Fase 5 — Estrazione campo per campo

Avvia vLLM con **Qwen3.8-27B** in AWQ 4-bit.

Raggruppa i campi in task: campi che condividono la stessa fonte nel documento vanno insieme, **massimo 5–8 per chiamata**. Per gli elementi di lista, un task per elemento.

Contesto ristretto: solo le regioni pertinenti + una regione di margine sopra e sotto. Mai il documento intero. Se non sai quali regioni siano pertinenti, aggiungi un passo di routing (BM25 + embedding sulle regioni, top-k); ma per moduli strutturati la posizione è di solito stabile e puoi ancorarla a un'etichetta.

Parametri: `temperature=0`, `seed` fisso, `guided_json=SchemaLoose` del task, backend `xgrammar`.

Struttura del prompt, in quest'ordine:

le regioni, ciascuna con il suo `region_id`;

la definizione dei campi da estrarre;

la regola che `quote` va copiata **carattere per carattere** dal testo fornito;

l'istruzione esplicita che `null` è la **risposta corretta** quando il dato non è presente.

Il punto 5.4 sul `null` non è pro forma: senza un'autorizzazione esplicita a non rispondere, il modello riempie il campo con la cosa più plausibile che trova nelle vicinanze.

Fase 6 — Verifica e validazione

Tre cancelli in sequenza. Un campo che ne fallisce uno non entra nell'output.

6.1 Cannello grounding

```
python
score = rapidfuzz.fuzz.partial_ratio(norm(quote), norm(page_full_text))
assert score >= 90
```

Sotto soglia il campo è allucinato: scartalo. La soglia è 90 e non 100 perché l'OCR introduce rumore; alzarla troppo produce falsi negativi su testo corretto ma letto imperfettamente.

6.2 Cannello coerenza value ↔ quote

Il valore deve essere **derivabile** dalla citazione:

numeri e date → estrai i token numerici dalla quote, verifica che `value` ne sia una normalizzazione;
enum → verifica che la quote contenga il termine che mappa a quel valore;
stringhe libere → verifica che `value` sia una sottostringa normalizzata della quote.

Questo cancello cattura il caso insidioso: **citazione autentica, interpretazione sbagliata**. È l'errore che il solo grounding non vede.

6.3 Cannello schema

`SchemaStrict.model_validate()`. Qui vivono i range, i formati e le regole cross-field.

6.4 Retry

Al fallimento di 6.1 o 6.3, rilancia la **stessa** chiamata con l'errore di validazione accodato al prompt. **Massimo 2 tentativi**. Al terzo, `status='needs_review'`.

Non ritentare all'infinito: un campo che fallisce tre volte ha un problema di dati o di prompt, non di campionamento.

6.5 Localizzazione

Cerca la quote nelle regioni della pagina, prendi la bbox della regione che la contiene, salva. Serve alla Fase 7.

Fase 7 — Revisione umana

Coda ordinata per rischio: prima i `confidence='low'`, poi i campi che hanno richiesto retry, poi i campi obbligatori risultati nulli.

UI minima: crop dell'immagine attorno alla bbox a sinistra, campo editabile a destra, accetta / correggi. Gradio basta — 100 righe.

Ogni correzione umana rientra nel gold set. Dopo qualche centinaio di documenti hai materiale per un fine-tuning, se i numeri lo giustificano.

Fase 8 — Valutazione

Una suite `pytest` che gira la pipeline sul gold set e produce, **per ogni campo dello schema:** precision, recall, exact match rate.

La metrica che conta più di tutte: tasso di errore silenzioso — campi con un valore sbagliato ma

`confidence='high'`. È l'unico errore che non intercetti, quindi l'unico che danneggia davvero la fedeltà. Un sistema che sbaglia molto ma lo dichiara è utilizzabile; uno che sbaglia poco ma in silenzio no.

Ogni run registra `git_sha`, versione del modello e versione dei prompt. Ogni modifica va confrontata con la run precedente sullo stesso set, altrimenti stai ottimizzando alla cieca.

Ordine di implementazione consigliato

Non costruire le fasi in ordine numerico. Costruisci prima lo scheletro end-to-end più corto che produce un JSON verificato, poi ispessiscilo:

Fasi 1 → 2 → 5 → 6.1 → 6.3 su un solo documento, con un sottoinsieme di 3 campi piatti. Obiettivo: vedere un JSON validato con citazione verificata.

Aggiungi la **Fase 8** (valutazione) *subito dopo*, prima di ogni altra cosa. Senza metriche, ogni miglioramento successivo è un'opinione.

Aggiungi la **Fase 4** (enumerazione liste) e la parte nidificata dello schema.

Aggiungi la **Fase 6.2** (coerenza value/quote).

Aggiungi la **Fase 3** (secondo OCR). È la più costosa in tempo di sviluppo e la sua utilità si misura solo se hai già le metriche del punto 2.

Aggiungi la **Fase 7** (revisione umana) quando il volume lo rende necessario.

Note sulla VRAM

PaddleOCR-VL-1.6 (0.9B): ~3GB.

Qwen3.8-27B AWQ 4-bit: ~18GB + KV cache.

Tecnicamente coesistono, ma la KV cache dell'estrattore vuole margine e con 32GB di RAM di sistema non hai spazio per l'offload. **Esegui le fasi in serie**, con vLLM che si spegne e riparte tra una fase e l'altra. Lo stato è su SQLite, quindi non perdi nulla.